BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 2 Practical No: 1

Roll no: 57 PRN No:12410559 Name of Student: Avni Selote

Subject Name : Deep Learning

Problem Statement :

Install and configure TensorFlow/Keras. Perform data preprocessing, normalization, train-test split, and visualization on a sample dataset.

Algorithm :

1. Start the program
2. Import the libraries : Tensorflow , Matplotlib .
3. Load the dataset from the Keras API . (I have used the fashion mnist dataset)
4. Preprocess the dataset : normalize the pixel values by dividing each pixel value by 255 to scale them to the range [0,1]
5. Check the dimensions of the training and the testing dataset
6. Visualize a sample image from the training dataset
7. Create a sequential Neural Network Model
8. Add a flatten layer to convert 28x28 input images into one dimesional vector
9. Add a Dense layer with 10 neurons and Softmax activation for multiclass classification
10. Compile the model . Use Accuracy as the evaluation metric
11. Train the model and normalize training dataset with specific number of epochs
12. Evaluate the trained model using testing dataset
13. Display accuracy
14. Save the model
15. Stop

Code & Output :

[3]:
```
import tensorflow as tf
import matplotlib.pyplot as plt
```

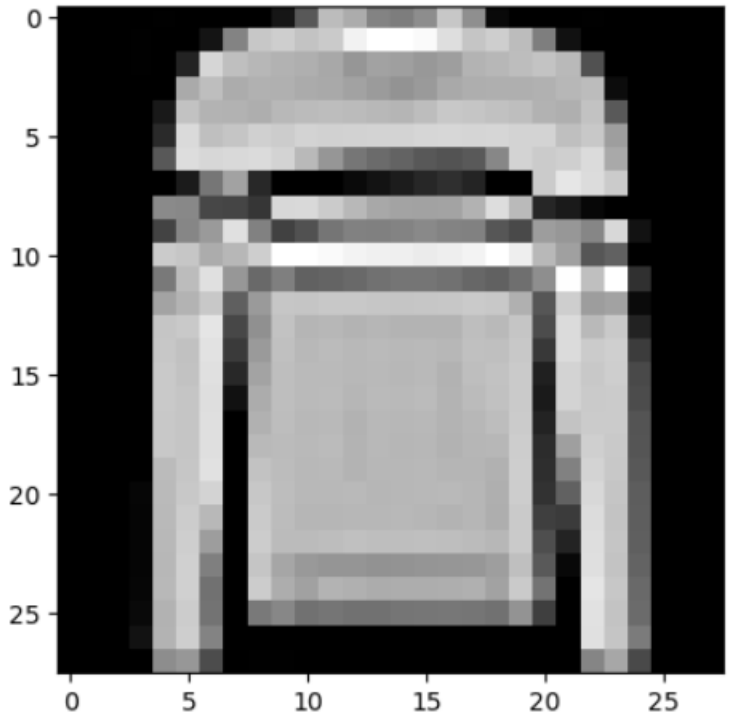
[4]:
```
(train_images, train_labels), (test_images, test_labels) = tf.keras.datasets.fashion_mnist.load_data()
```

[5]:
```
train_images = train_images /255.0
test_images  = test_images  /255.0
```

[6]:
```
print(train_images.shape)
print(test_images.shape)
print(train_labels)
```

```
(60000, 28, 28)
(10000, 28, 28)
[9 0 0 ... 3 0 5]
```

[11]:
```
plt.imshow(train_images[5], cmap='gray')
plt.show()
```



```
my_model = tf.keras.models.Sequential()
my_model.add(tf.keras.layers.Flatten(input_shape=(28,28)))
my_model.add(tf.keras.layers.Dense(128, activation='relu'))
my_model.add(tf.keras.layers.Dense(10, activation='softmax'))

my_model.compile(optimizer='adam' , loss='sparse_categorical_crossentropy' , metrics=['accuracy'])

my_model.fit(train_images, train_labels, epochs=3)
```

```
Epoch 1/3
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 3s 1ms/step - accuracy: 0.8242 - loss: 0.4995
Epoch 2/3
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 2s 1ms/step - accuracy: 0.8651 - loss: 0.3763
Epoch 3/3
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 2s 1ms/step - accuracy: 0.8770 - loss: 0.3374
<keras.src.callbacks.history.History at 0x17fac8ac2d0>
```

]:
```
val_loss, val_acc = my_model.evaluate(test_images, test_labels)
print('Test accuracy : ', val_acc)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - accuracy: 0.8581 - loss: 0.3941
Test accuracy :  0.8580999970436096
```

]:
```
my_model.save('model.keras')
```